

Pai Theory Assignment#1

Arshia Azeem 24k-0012 BSAI-3A

Task1:

```
import itertools
```

```
transactionLog=[  
    {'orderId':1001,'customerId':'cust_Ahmed','productId':'prod_10'},  
    {'orderId':1001,'customerId':'cust_Ahmed','productId':'prod_12'},  
    {'orderId':1002,'customerId':'cust_Bisma','productId':'prod_10'},  
    {'orderId':1002,'customerId':'cust_Bisma','productId':'prod_15'},  
    {'orderId':1003,'customerId':'cust_Ahmed','productId':'prod_15'},  
    {'orderId':1004,'customerId':'cust_Faisal','productId':'prod_12'},  
    {'orderId':1004,'customerId':'cust_Faisal','productId':'prod_10'},  
]
```

```
productCatalog={  
    'prod_10':'Wireless Mouse',  
    'prod_12':'Keyboard',  
    'prod_15':'USB-C Hub'  
}
```

```
def processTransactions(transactionList):  
    cust_prod={}  
    for record in transactionList:  
        cust=record['customerId']  
        prod=record['productId']
```

```
    if cust not in cust_prod:
        cust_prod[cust]=set()
    cust_prod[cust].add(prod)
return cust_prod
```

```
def findFrequentPairs(customerData):
    pair_counts={}
    for cust, prod_set in customerData.items():
        if len(prod_set)<2:
            continue
        for p1,p2 in itertools.combinations(prod_set,2):
            pair=tuple(sorted((p1,p2)))
            if pair in pair_counts:
                pair_counts[pair]+=1
            else:
                pair_counts[pair]=1
    return pair_counts
```

```
def getRecommendations(targetProductId,frequentpairs):
    related={}
    for pair,count in frequentpairs.items():
        if pair[0]==targetProductId:
            if pair[1] not in related:
                related[pair[1]]=count
            else:
```

```

        related[pair[1]]+=count
elif pair[1]==targetProductId:
    if pair[0] not in related:
        related[pair[0]]=count
    else:
        related[pair[0]]+=count
return related

```

```

def generateReport(targetProductId,recommendations,catalog):
    print(f'Recommendations for:{catalog[targetProductId]}')
    if not recommendations:
        print("No related products")
        return
    sorted_rec=sorted(recommendations.items(),key=lambda x:x[1],reverse=True)

    product_ids,count=zip(*sorted(recommendations.items(),key=lambda
x:x[1],reverse=True))
    product_names=[catalog[p] for p in product_ids]
    for i,(name,count) in enumerate(zip(product_names,count),start=1):
        print(f'{i}. {name} - {count}')

```

```

transactions=(processTransactions(transactionLog))
print("transformed Data:\n",transactions)
pairCount=findFrequentPairs(transactions)
print("Pairs:\n",pairCount)

```

```
recommendations=getRecommendations('prod_10',pairCount)
print("Recommendations:\n",recommendations)
generateReport('prod_10',recommendations,productCatalog)
```

Task2:

```
import string

def preprocessText(text,punctuationList,stopwordsSet):
    text=text.lower()
    text="".join([c for c in text if c not in punctuationList])
    text=' '.join([w for w in text.split() if w not in stopwordsSet])
    return text

def analyzePosts(postsList,punctuationList,stopwordsSet,positive,negative):
    processed=[]
    for p in postsList:
        clean_text=preprocessText(p['text'],punctuationList,stopwordsSet)
        s=0
        for w in clean_text.split():
            if w in positive:s+=1
            elif w in negative:s-=1
        processed.append({'id':p['id'],'text':p['text'],'score':s})
    return processed

def getFlaggedPosts(scoredPosts,sentimentThreshold=-1):
    return [p for p in scoredPosts if p['score']<=sentimentThreshold]

def findNegativeTopics(flaggedPosts):
    negTopics={}
    for p in flaggedPosts:
        negTopics[p['text']] = p['score']
```

```

for p in flaggedPosts:
    for w in p['text'].split():
        if w.startswith('#') or w.startswith('@'):
            tag="".join([c for c in w if c.isalnum() or c in ['#','@']])
            if tag not in negTopics:negTopics[tag]=1
            else:negTopics[tag]+=1
    return negTopics

allPosts=[
    {'id':1,'text':'I LOVE the new #GuLPhone! Battery life is amazing.'},
    {'id':2,'text':'My #GuLPhone is a total disaster. The screen is already broken!'},
    {'id':3,'text':'Worst customer service ever from @GuLPhoneSupport. Avoid this.'},
    {'id':4,'text':'The @GuLPhoneSupport team was helpful and resolved my issue. Great
service!'}
]

PUNCTUATION_CHARS='!"#$%%&'()*+,-./:;<=>?@[\\]^_`{|}~"'

STOPWORDS_SET={'i','me','my','a','an','the','is','am','was','and','but','if','of','to','at','by','for','with','this',
'that'}

POSITIVE_WORDS_SET={'love','amazing','great','helpful','resolved'}

NEGATIVE_WORDS_SET={'disaster','broken','worst','avoid','bad'}

processed=analyzePosts(allPosts,PUNCTUATION_CHARS,STOPWORDS_SET,POSITIVE_W
ORDS_SET,NEGATIVE_WORDS_SET)

flagged=getFlaggedPosts(processed,sentimentThreshold=-1)

topics=findNegativeTopics(flagged)

print("Flagged Posts:",flagged)

print("Negative Topics:",topics)

```

Task3:

```
class Package:
```

```
    def __init__(self,packageId,weightInKg):  
        self.packageId=packageId  
        self.weightInKg=weightInKg
```

```
class Drone:
```

```
    def __init__(self,dronId,maxLoadInKg):  
        self.dronId=dronId  
        self.maxLoadInKg=maxLoadInKg  
        self._status='idle'  
        self.currentPackage=None  
        self.deliveryTimer=0
```

```
    def getStatus(self):  
        return self._status
```

```
    def setStatus(self,newStatus):  
        if newStatus in ['idle','delivering','charging']:  
            self._status=newStatus
```

```
    def assignPackage(self,packageObj):  
        if self._status=='idle' and packageObj.weightInKg<=self.maxLoadInKg:  
            self.currentPackage=packageObj  
            self.setStatus('delivering')
```

```
    self.deliveryTimer=2  
  
    return True  
  
return False
```

```
class FleetManager:
```

```
    def __init__(self):
```

```
        self.drones={}
```

```
        self.pendingPackages=[]
```

```
    def dispatchJobs(self):
```

```
        for p in self.pendingPackages[:]:
```

```
            for d in self.drones.values():
```

```
                if d.getStatus()=='idle':
```

```
                    if d.assignPackage(p):
```

```
                        self.pendingPackages.remove(p)
```

```
                        break
```

```
    def simulationTick(self):
```

```
        for d in self.drones.values():
```

```
            if d.getStatus()=='delivering':
```

```
                d.deliveryTimer-=1
```

```
            if d.deliveryTimer<=0:
```

```
                d.setStatus('charging')
```

```
                d.currentPackage=None
```

```
            elif d.getStatus()=='charging':
```

```
d.setStatus('idle')
```

```
fm=FleetManager()
```

```
d1=Drone("D1",10)
```

```
d2=Drone("D2",5)
```

```
fm.drones[d1.droneld]=d1
```

```
fm.drones[d2.droneld]=d2
```

```
p1=Package("P1",4)
```

```
p2=Package("P2",8)
```

```
p3=Package("P3",6)
```

```
fm.pendingPackages=[p1,p2,p3]
```

```
for i in range(5):
```

```
    fm.dispatchJobs()
```

```
    fm.simulationTick()
```

```
    for d in fm.drones.values():
```

```
        print(d.droneld,d.getStatus())
```

```
    print()
```

Task4:

```
class Image:
```

```
    def __init__(self,pixelData):
```

```
        self.pixelData=pixelData
```

```
    def getCopy(self):
```

```
        return [row[:] for row in self.pixelData]
```

```
def applyTransformation(self,transformationFunc):  
    self.pixelData=transformationFunc(self.pixelData)
```

```
def flipHorizontal(pixelData):  
    return [row[::-1] for row in pixelData]
```

```
def adjustBrightness(pixelData,brightnessValue):  
    return [[pixel+brightnessValue for pixel in row] for row in pixelData]
```

```
def rotateNinetyDegrees(pixelData):  
    rows=len(pixelData)  
    cols=len(pixelData[0])  
    return [[pixelData[rows-1-r][c] for r in range(rows)] for c in range(cols)]
```

```
class AugmentationPipeline:  
    def __init__(self):  
        self.steps=[]  
  
    def addStep(self,transformFunc):  
        self.steps.append(transformFunc)  
  
    def processImage(self,originalImage):  
        results=[]  
        for func in self.steps:
```

```
imgCopy=Image(originalImage.getCopy())  
imgCopy.applyTransformation(func)  
results.append(imgCopy.pixelData)  
return results
```

```
originalPixels=[[10,20,30],[40,50,60]]
```

```
img=Image(originalPixels)
```

```
pipe=AugmentationPipeline()
```

```
pipe.addStep(flipHorizontal)
```

```
pipe.addStep(lambda data:adjustBrightness(data,10))
```

```
pipe.addStep(rotateNinetyDegrees)
```

```
augmented=pipe.processImage(img)
```

```
for a in augmented:
```

```
    print(a)
```